

COMS W4156 Advanced Software Engineering (ASE)

September 19, 2023



Attendance



We start taking attendance **TODAY** using zoom's 'usage reports'

These show each attendee's zoom name, join time, leave time, and duration for everyone who attended that zoom session

Make sure your zoom name matches your courseworks name, or we will not be able to give you credit for attending. If there is some reason you cannot make them match you need to explain (by posting a private message to the teaching staff, not just me, in edstem)

Make sure you stay for the whole session unless you tell us (by posting a private message to the teaching staff, not just me, in edstem) why you had to arrive late and/or leave early

If you have a bad connection and you manually disconnect and reconnect, or zoom disconnects you and you reconnect, we can put the two (or more) together as long as it's the same zoom name

Looking for Group

If anyone does not yet have a team, or has a team with less than four or more than five members, please speak up NOW



To Be Filled In On The Fly

Agenda

1. Looking for Group
2. **RESTful APIs**
3. Express/Node.js demo: Tom Le
4. Spring/Java demo: Tan Nguyen

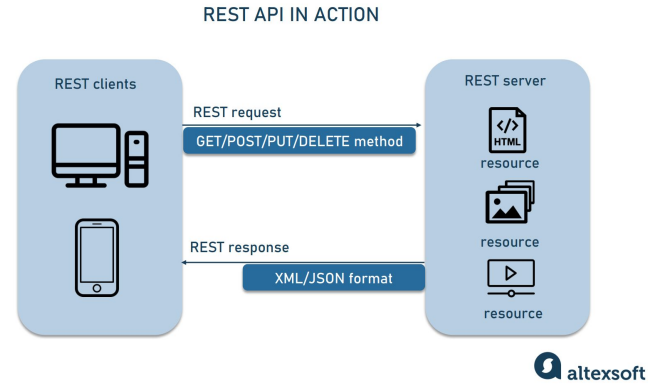


RESTful APIs

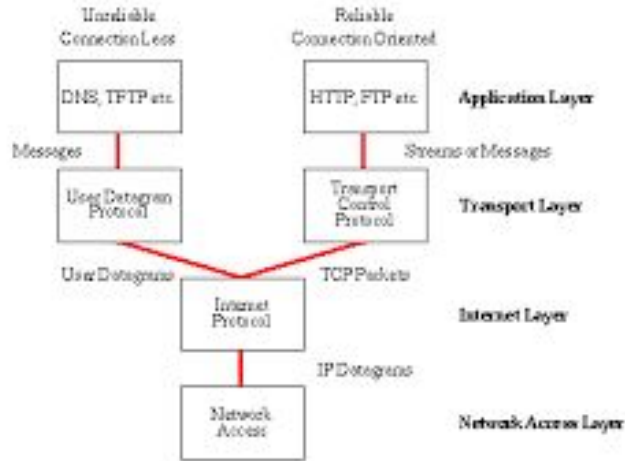
RESTful API and REST API are the same thing,
“RESTful” is the technically correct term

A RESTful API is usually accessed remotely
via [HTTP](#) (HyperText Transfer Protocol) or
[HTTPS](#) (Secure HTTP or HTTP with Security)
over the Internet, with a predefined set of
[URLs](#) (Uniform Resource Locators) that
constitute the API

HTTP or HTTPS is the *protocol*,
not itself the API or part of the API



Protocols vs. APIs



Protocol = the rules for initiating, continuing and completing communications among software components and how the communication messages are formatted in terms of bytes (e.g., [SSH](#) is another protocol you have probably used)

API (Application Programming Interface) = the set of function calls or other entry points that a given component understands (e.g., [Java SE and JDK](#), [Facebook APIs](#))

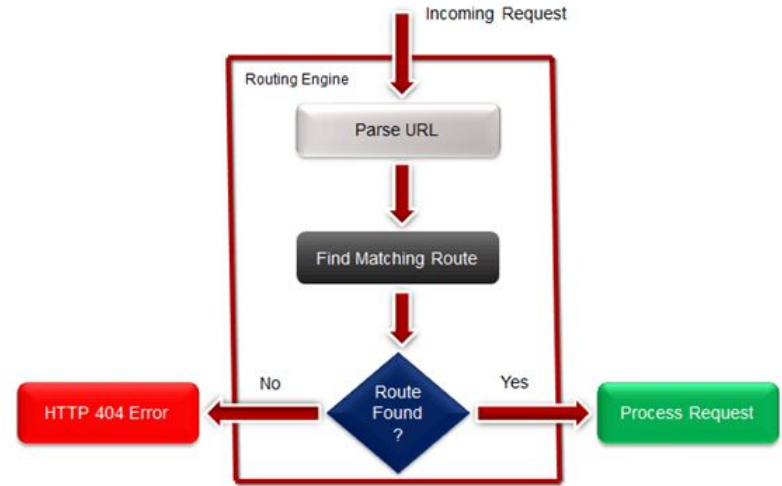
We are concerned in this class primarily with APIs, but you will need to use some protocol to send API calls across processes and machines

Routes and Endpoints

In the RESTful context, a URL is often called a “[route](#)”, the domain and path tell the client (and any intermediate internet nodes) how to get to the place where the service runs. The path looks like a file system path, but can be interpreted in some other way by the domain

The “[endpoint](#)” is the final part of the URL after the domain and path, telling the service which entry point is being called,
`http://example.com/very/long/path/to/where/is/the/endpoint`

HOW ROUTING WORKS



RESTful API Methods

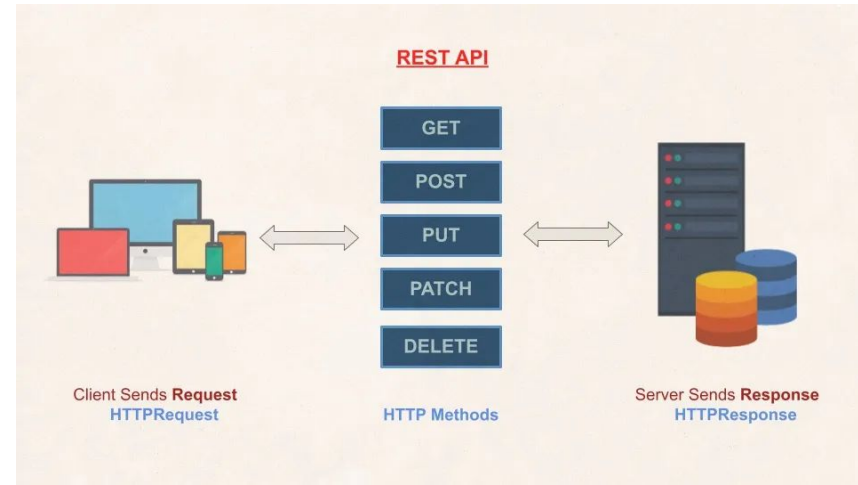
A RESTful API consists of a set of HTTP method calls:

HTTP METHOD endpoint1 { optional body with data }

...

HTTP METHOD endpointN { optional body with data }

where “endpointN” corresponds to the API function to call with the data parameters



RESTful API terminology uses the term “method” to refer to the HTTP operations to apply to a given endpoint, typically at least GET and POST (possibilities listed [here](#) plus new one [PATCH](#))

Simplest RESTful API Calls



Consider the URL `http://example.com/very/long/path/to/where/is/the/endpoint`

If you click a link like this one on a page in your browser, the browser will use HTTP to connect to the web server at `example.com` (domain) and tell it to access the `/very/long/path/to/where/is/the/endpoint` file directory path from the webroot

A client program that is not a browser can also use HTTP to connect to a service, that is not a web server, at `example.com` (domain) and tell it to handle `/very/long/path/to/where/is/the/endpoint`

In the web case, this typically fetches some content located at `/very/long/path/to/where/is/the/endpoint`, which is then returned to and rendered by your browser. The 'rendering' could give you text, image, audio, video, code, or any other data

But this could instead run a program denoted by `/very/long/path/to/where/is/the/endpoint` and then return the results of running that program, with the results (not the program) returned to the client to process

What do RESTful API calls return?

REST = [REpresentational State Transfer](#) because accessing the resource *transfers* to the caller a *representation* of the current *state* of the resource (nowadays usually [XML](#) or [JSON](#) for data)

The HTTP caller (client) can be any program, the requestor does not need to be a web browser

And the callee can be any program, the response does not have to come from a web service

When the HTTP caller is your program,
you are using a RESTful API

When the HTTP callee is your program,
you are implementing a RESTful API



What is a “Representation”?

The same resource (or resource state) may have many different representations (variants).
For just a plain static web page,

- A page of text might be presented in HTML or plain text
- The words in the text might be written in English or Chinese
- The characters in the text might be encoded in ASCII or UTF-8
- The page might be organized differently for a PC versus a mobile device

The client might specify the representation(s) it can handle by including optional fields in the HTTP header or the server might send multiple representations and the client uses one or more of those it can handle

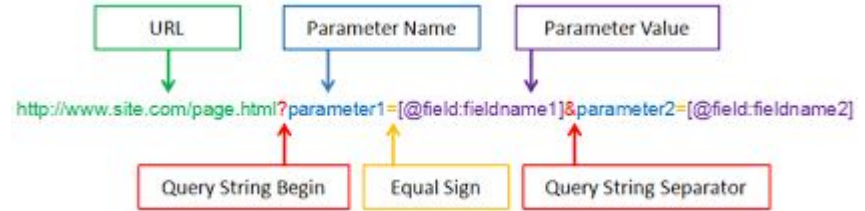
Determining what to send/receive is called [content negotiation](#)



RESTful APIs with Basic Parameters

`http://example.com/path/to/resource?query`

A query string looks something like
`field1=value1&field2=value2&field3=value3`



If you click a query URL like this one in your browser, the browser will use HTTP to connect to the web server at `example.com`, tell the program at the `/path/to/resource` endpoint to run, and give it the parameters `value1`, `value2` and `value3`. The browser then renders the results of running that program with the parameters

If a client program constructs a query like this one and uses HTTP to send it to the service at `example.com/path/to/resource`, with the parameters `value1`, `value2` and `value3`, the service program usually runs the designated function with those parameters and returns the result to the client for processing. But the service might do something else, such as send the result to a third party, according to the semantics of the API function

Where does the Query come from?

Usually a query URL would not be sitting there embedded in a page (or in the client code) with the field values already filled in

Instead the values of the fields would likely be computed somehow, in the browser case possibly by showing the user a form to fill in - then the form displays the fields and the user enters the values

Again the HTTP caller client can be any program, it does not need to be a web browser, and the callee service can be any program, it does not need to be a web service

How do I write my name?

First Name (Last)		Middle Name		Last Name	
Date of Birth (MM/DD/YYYY)		Applicant Signature			
Complete this form carefully. Please print.					
First Name		Last Name		Middle Name	
Date of Birth		Day	Month	Year	Current Age
Complete this form carefully. Please print.					
FIRST NAME		LAST NAME		MIDDLE NAME	
DATE OF BIRTH (MONTH, DAY, YEAR)		SIGNATURE			
1. Print your name in all caps on your card or business card.					
2. Print your name in all caps on your card or business card.					
3. Print your signature.					

When the caller /callee is your program, you are again using/implementing a RESTful API

Where are the GET and POST?

A URL like `http://example.com/path/to/resource` would presumably not change the state of the resource (except maybe to increment [metadata](#) like how many times the page has been accessed or update the timestamp for the most recent access)

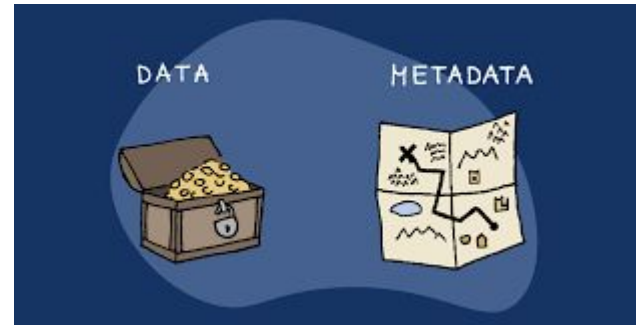
If your API call only intends to fetch something with no side-effects on the resource state, use [GET](#)

A URL query like

`http://example.com/path/to/resource?field1=value1&field2=value2&field3=value3`

also should not have side-effects

If your API call only intends to fetch something, e.g., the field values will be used for search with no side-effects on the resource state, then again use GET - in this case with the query fields/values included in the HTTP header



Many Services Have State

 Stateless	 Stateful
• Server requests based on information with each request	• Based on internet protocol which requires a tight state
• Does not rely on the earlier requests information	• Requests based on the information relayed with each request
• Server does not hold requests information	• Information stored from earlier requests
• Different servers can different information at once	• Same server must be used to process all requests



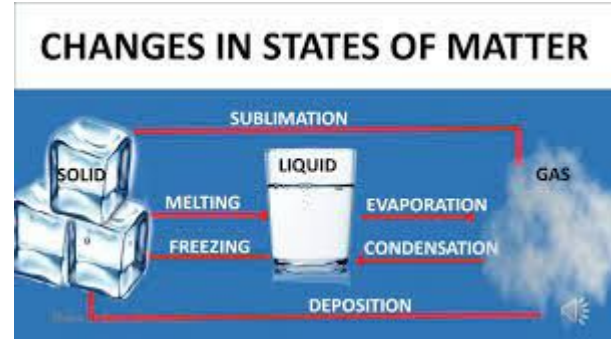
Let's say we wanted to update some data to set `field1=value1&field2=value2&field3=value3`

That is, your API call intends to *change* the state of the resource based on the provided field values, then use [POST](#) and put the fields/values in the body of the HTTP request rather than the header

That is, `http://example.com/path/to/resource` goes in the http *header* and the query string `field1=value1&field2=value2&field3=value3` goes in the http *body* rather than the header

Many Services Compute State

Let's say we wanted to update some data based on the result of running the resource program with parameters `field1=value1&field2=value2&field3=value3`



Again, if the API call is intended to *change* the state of the resource based on the provided field values, then the API should expect the POST method and expect to find the fields/values in the body of the HTTP request rather than the header (and might produce an [error status code](#) otherwise)

Again `http://example.com/path/to/resource` goes in the http header and the query string `field1=value1&field2=value2&field3=value3` goes in the http body rather than the header

The code that implements the client caller and service endpoint is just like any other code you might write, except it probably uses [json](#), [xml](#), or some other [easily serializable format](#) for representing data

Agenda

1. Looking for Group
2. RESTful APIs
3. Express/Node.js demo: Tom Le
4. Spring/Java demo: Tan Nguyen



Express/Node.js demo: Tom Le

<https://github.com/lequangtri20/node-server>

Agenda

1. Looking for Group
2. RESTful APIs
3. Express/Node.js demo: Tom Le
4. Spring/Java demo: Tan Nguyen



Spring/Java demo: Tan Nguyen

<https://github.com/tanmnguyen/ExampleSpringApplication>

Upcoming Assignments



[Team Formation](#) was due yesterday, but you can still (re-)submit

Teams who grew, shrank, or otherwise changed should re-submit

Students who have a team now but didn't yesterday, or vice versa, should re-submit

[Team Preliminary Project Proposal](#) due next Monday, September 25, by 11:59pm

[Team Revised Project Proposal](#) due a week later Monday, October 2, by 11:59pm

Thursday

RPC-based APIs with gRPC demo

We will return to (execution-based) bug finding next week

 thrifty.io protocol comparison	First released	Formatting type	Key strength
SOAP	Late 1990s	XML	Widely used and established
REST	2000	JSON, XML, and others	Flexible data formatting
JSON-RPC	mid-2000s	JSON	Simplicity of implementation
gRPC	2015	Protocol buffers by default; can be used with JSON & others also	Ability to define any type of function
GraphQL	2015	JSON	Flexible data structuring
Thrift	2007	JSON or Binary	Adaptable to many use cases

Reminder: Attendance



We start taking attendance **TODAY** using zoom's 'usage reports'

These show each attendee's zoom name, join time, leave time, and duration for everyone who attended that zoom session

Make sure your zoom name matches your courseworks name, or we will not be able to give you credit for attending. If there is some reason you cannot make them match you need to explain (by posting a private message to the teaching staff, not just me, in edstem)

Make sure you stay for the whole session unless you tell us (by posting a private message to the teaching staff, not just me, in edstem) why you had to arrive late and/or leave early

If you have a bad connection and you manually disconnect and reconnect, or zoom disconnects you and you reconnect, we can put the two (or more) together as long as it's the same zoom name

Ask Me Anything